

Fibonacci Heap Implementation and Empirical Comparison with an Indexed Binary Heap

1. What I Built

1.1 Project Scope and Supported Operations

For this assignment, I implemented a Fibonacci min-heap in C++17. As per the assignment requirements, I also chose Track A: Implementation and Empirical Study. Not only did I implement a working data structure, but I also came up with the code for a different data structure (Indexed Binary Heap). The implemented Indexed Binary Heap is designed to have similar features on the surface, such as insertion, minimum-lookup, extraction, decrease-key, size, and emptiness checks, whilst also differing in internal structures. Algorithms to benchmark and test these two heaps using the same inputs as fairly as possible are also derived. Then, I used the resulting benchmark data and constructed an Empirical Study as I analyzed them. However, for both heaps, melding and arbitrary deletion were considered out of scope since they are not tested and exercised in the empirical study.

1.2 Data Representation and Key Design Decisions

Within my implementation, the Fibonacci heap consists of nodes that utilize parent, child, and circular sibling pointers along with degree and marking information. A minimum pointer (`min_`) inside the Fibonacci heap identifies its smallest root. Opaque handles are used in the heap as well to identify items without exposing the nodes and their identification numbers. On the other hand, in the indexed binary heap implemented, an entry vector and an ID-to-position vector were used. For both heaps, copying is disabled to avoid ownership and handle-semantics complications. Extracted handles become invalid, and handle IDs are not reused.

1.3 Code Organisation

While the repository consists of multiple folders, each folder stores different file types.

- `include/` and `src/` contain interfaces and implementations.
- `tests/` contains correctness checks.
- `benchmarks/` contains algorithms for the three experiments.
- `resultsUsedInReport/` preserves the measurements behind the figures used in this report.
- `results/` stores fresh measurements and is overwritten during every default CMake build.
- `report/` stores this current report file.

2. Empirical Study

2.1 Baseline and Hypothesis

For this assignment, I chose the Indexed Binary Heap as the baseline. This is mainly because it supports the same operations as the Fibonacci Heap, including decrease-key via handles, making it a suitable candidate for comparison. When it comes to the decrease-key function, the Fibonacci heap has $O(1)$ amortized time complexity, while the Binary Heap has $O(\log n)$ time complexity in the worst case. On the other hand, extraction takes $O(\log n)$ amortized time for the Fibonacci heap and $O(\log n)$ worst-case time for the indexed binary heap. Hence, the Fibonacci Heap isn't guaranteed to beat the Binary heap in insert-extract.

Initial hypothesis: I expected the Fibonacci heap to perform decreases faster than the indexed binary heap, particularly as the update ratio increased from N to $10N$. I also expected that these savings might reduce the total workload time.

Revised hypothesis: After examining the small-decrease results, I expected decreases that make each target the new minimum to expose a larger Fibonacci advantage during the update phase, because the binary heap would need more upward movement.

2.2 Experimental Environment

The same compiler and optimisation settings were used to compile both heaps together. In this report, however, the fixed CSVs saved in the resultsUsedInReport/ folder were used for evaluation, and the results are subject to change on different runs according to different experimental environments and hardware capabilities.

Component	Setting
Compiler	Apple Clang 21.0.0
Architecture	ARM64
Operating System	macOS 26.5.1
C++ standard	C++17
Build configuration	CMake Release
CMake version and generator	4.4, Unix Makefiles

Compiler flags	-O3 -DNDEBUG -std=c++17 -arch arm64 -Wall -Wextra -Wpedantic
Timer	std::chrono::steady_clock
Timing unit	Milliseconds
Processor Model	Apple M3 Pro
RAM	18 GB unified memory

2.3 Experimental Design

For all experiments, the workload is defined by the following setup

- N = 1000, 10000, and 100000
- Seeds = 42, 123, and 2026, using std::mt19937

During each experiment, both heaps received equal inputs and update sequences.

2.3.1 Insert–Extract

For this experiment, the following sequence is practised.

1. Generate N random integer keys between -1,000,000 and 1,000,000 inclusive.
2. Insert all N keys.
3. Extract all nodes in the heap until it is empty.
4. Compare the extraction sequence against a sorted copy of the input.

For this experiment, no decrease-key operations were called.

2.3.2 Small Decreases

For this experiment, the following sequence is practised.

1. Generate N initial keys between 1,000,000 and 2,000,000.
2. Insert sentinel 0 and the N items, retaining the items' handles.
3. Extract the sentinel to trigger Fibonacci consolidation before updates.
4. Apply N or 10N updates, choosing targets randomly.
5. Each update subtracts 1-100 (at random) from the target's latest key, ensuring a strict decrease.
6. The heap is drained afterward.

In this experiment, not every decrease-key operation is guaranteed to perform an expensive operation; i.e., a cut operation in a Fibonacci Heap and an upward swap in a Binary Heap. This is because it is not always guaranteed that the new key input into the decrease_key function is less than its parent.

Additionally, a separate vector is also utilized to track each item's latest key after every update during workload generation. Then, the final keys in this vector are sorted to form the expected extraction

sequence. After the heap is drained, all of its output is compared against this sequence for a correctness check.

2.3.3 Large Decreases

For this experiment, the following sequence is practised.

1. Generate N initial keys between 1,000,000 and 2,000,000.
2. Insert sentinel 0 and the N items, retaining the items' handles.
3. Extract the sentinel to trigger Fibonacci consolidation before updates.
4. Apply N or 10N updates, choosing targets randomly. The newly assigned keys are -1, -2, -3, ... in sequence.
5. Since every update makes its target the new minimum, the binary upward movement and Fibonacci cuts are encouraged.

In this experiment, it is still not guaranteed that every `decrease_key` operation will encounter binary upward movement or Fibonacci cuts. A target at the root needs no upward movement or cut.

As in the previous experiment, correctness is checked afterward, where the extracted keys are compared against the sorted final keys after all recorded updates.

In both experiments 2.3.2 and 2.3.3, either N or 10N describes the total number of `decrease_key` operations received by each heap. It does not reflect the number of `decrease_key` operations received by each node. Moreover, the `decrease_key` process is done in both of these processes. Insertion and extraction operations are not carried out between `decrease_key` operations.

2.4 Timing and Repetitions

In all experiments mentioned before, `std::chrono::steady_clock` is utilized to measure time in milliseconds. In experiment 2.3.1, the time taken to do all insertion and extraction operations is measured. However, in experiments 2.3.2 and 2.3.3, the time taken to do all `decrease_key` operations is recorded separately from the total time taken to do all operations, including insertion and extraction. However, input generation, reference sorting, output allocation, validation, and heap destruction are not included in the time measurement in any of the experiments.

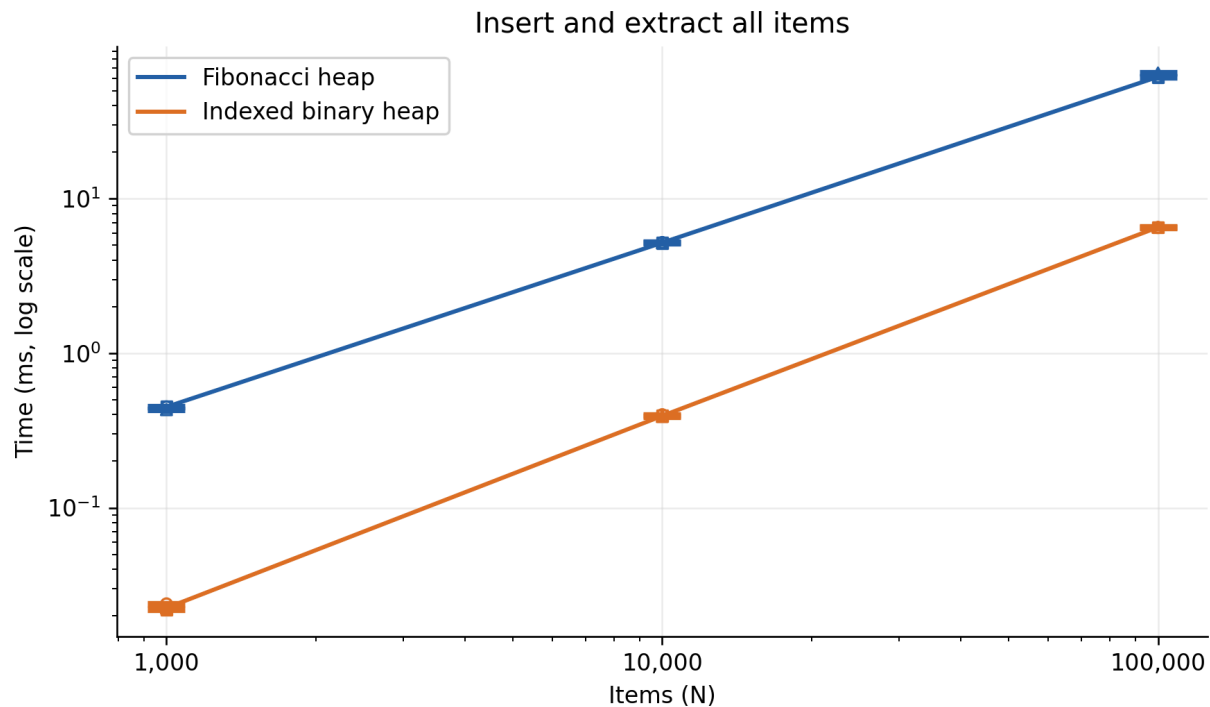
Every experiment starts with a discarded warm-up per heap for each configuration that is not counted in the time measured. For the same configurations, there are 5 measured repetitions for each heap, and the order of the runs is alternated between each repetition to reduce consistent ordering bias.

2.5 Results

After receiving all time measurements from the experiments in Section 2.3, the median of the five repetitions for each seed and configuration is calculated. Then, the median of the three seeds' medians is selected. This value is later injected into Matplotlib in Python to derive a plotted points chart. On these plotted point charts, whiskers are included to portray the minimum-maximum seed medians, but not confidence intervals. The individual marks show each seed's median, and the connecting line shows the

median of those three medians. In total, there are 450 measured runs, of which 90 are in experiment 2.3.1 and 180 each for both experiments 2.3.2 and 2.3.3.

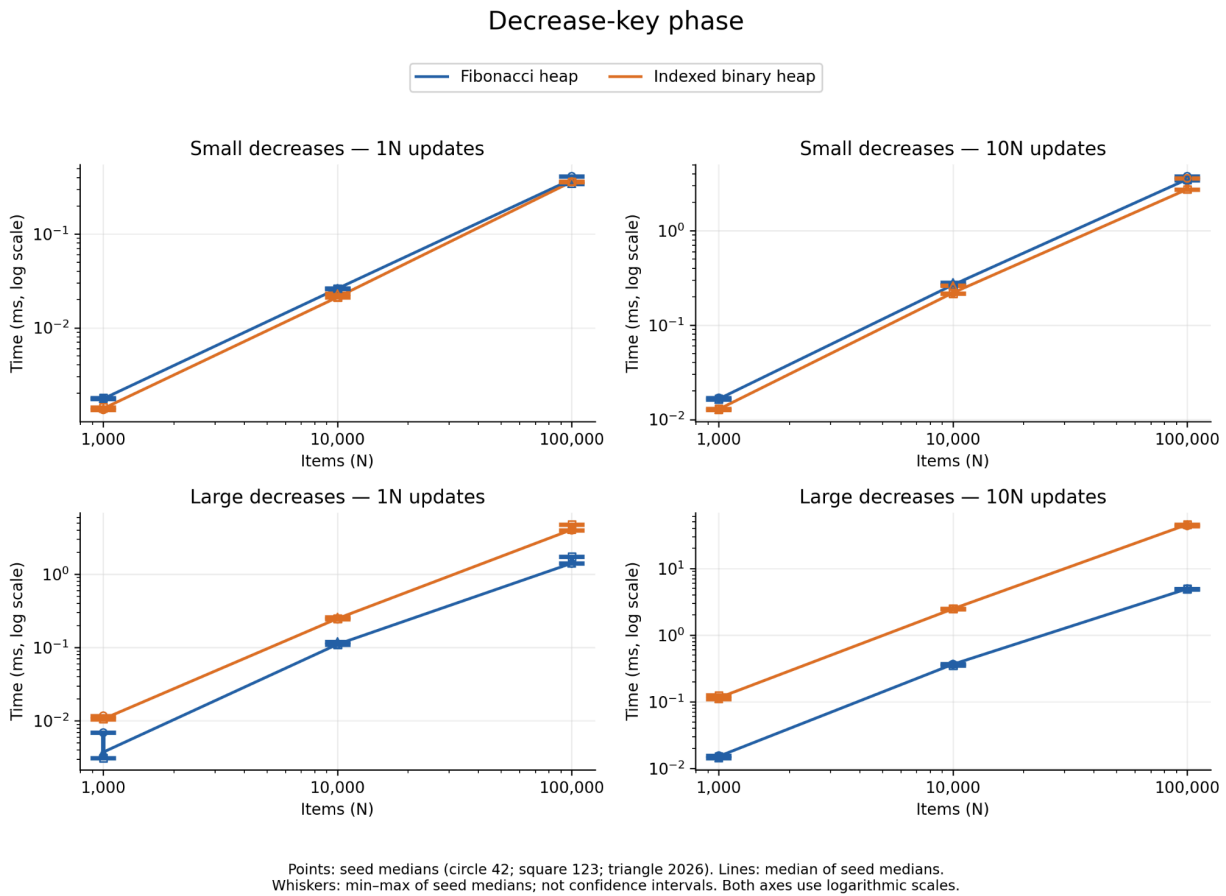
2.5.1 Insert–Extract Performance



Points: seed medians (5 repetitions each). Lines: median of seed medians.
Whiskers: min-max of seed medians; not confidence intervals.

In this chart, it is apparent that the Indexed Binary Heap is faster than the Fibonacci Heap in all three tested sizes. Additionally, it can also be observed that both heaps take longer to perform insertion and extraction operations as N increases. From checking the direct data produced in the .csv files, it can be seen that when $N=100,000$, the Fibonacci heap took 61.352 ms while the Binary Heap took 6.528 ms, which is about 9.4 times longer. In this experiment, there are no `decrease_key` operations, so this result shouldn't reflect the Fibonacci heap's main decrease-key advantage.

2.5.2 Decrease-Key Performance



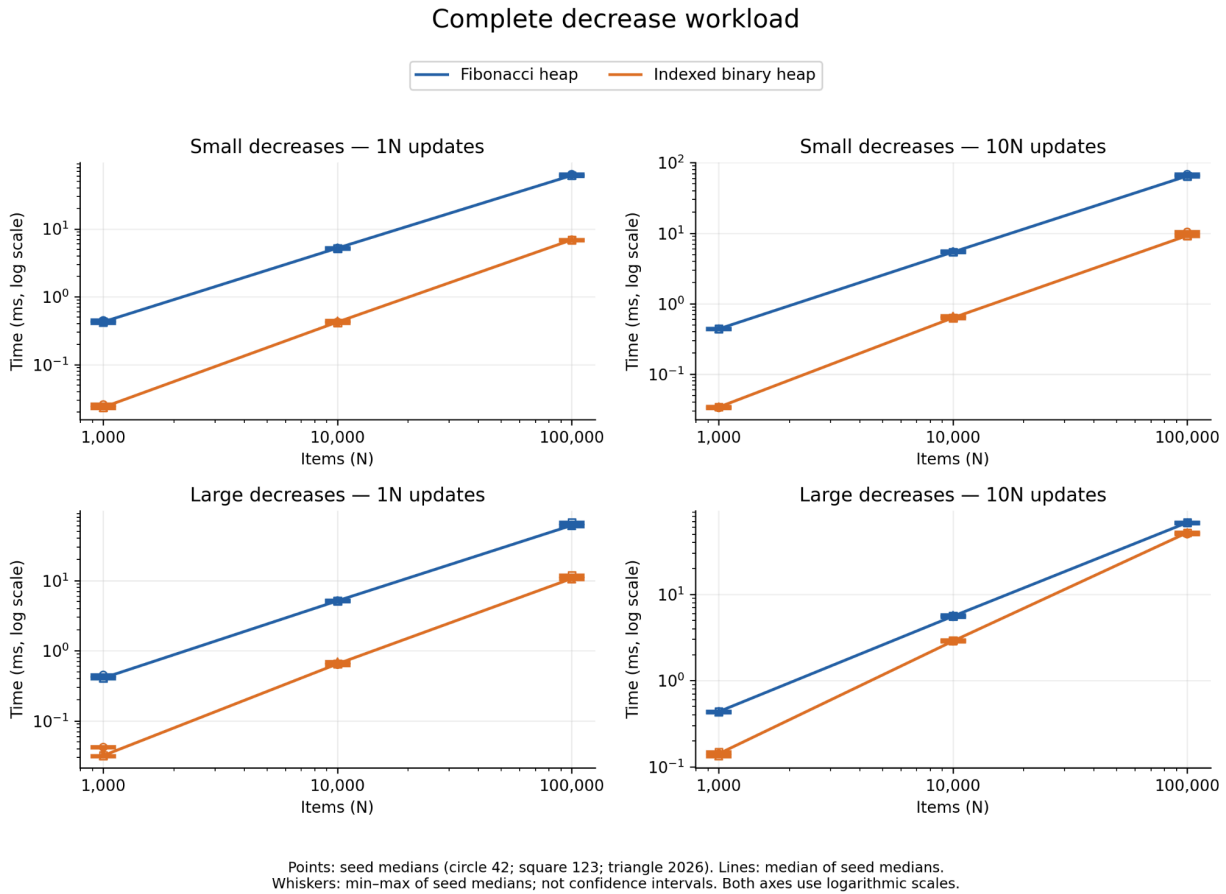
CSV results:

At $N = 100,000$,

Workload	Updates	Fibonacci updates	Binary updates
Small decreases	N	0.383 ms	0.362 ms
Small decreases	10N	3.518 ms	2.756 ms
Large decreases	N	1.417 ms	4.075 ms
Large decreases	10N	4.949 ms	45.519 ms

By looking at these charts, it is apparent that small decrease_key operations (top row) favour binary at both update quantity ratios, going against my initial hypothesis. However, when we look at the large decrease_key operations (bottom row), it is apparent that the Fibonacci heap is favored. After checking the CSV files and deriving the medians, it can be seen that the Fibonacci heap is 2.9 times faster at N updates and 9.2 times faster at 10N updates, supporting my revised hypothesis. It can also be observed that increasing the update count for both heaps from N to 10N results in an increased update time for both heaps.

2.5.3 Complete Workload Performance



CSV results:

At N = 100,000,

Workload	Updates	Fibonacci total	Binary total
Small decreases	N	61.360 ms	6.901 ms
Small decreases	10N	64.754 ms	9.232 ms
Large decreases	N	61.206 ms	10.683 ms
Large decreases	10N	68.174 ms	52.067 ms

In all tested configurations, the Binary heap remains faster than the Fibonacci Heap for the complete workload performance, even though the Fibonacci Heap won the test for the large-decrease update phase. When we compare the results from the CSV files, it can also be calculated that at 100,000 items and 10N large decreases, the complete workload of the Fibonacci Heap takes about 1.31 times longer than the Binary Heap, even though the Fibonacci's updates are 9.2 times faster. When looking at the graph, it is also apparent that the overall gap, however, narrows as large decreases become more frequent, since at N updates, Fibonacci takes about 5.7 times longer, compared with 1.31 times at 10N. Hence, we can

conclude that faster `decrease_key` operations do not automatically positively affect the results for the speed of the complete workload.

2.6 Interpretation and Limitations

2.6.1 Interpretation

For a Binary Heap, small `decrease_key` operations may preserve parent order, stopping the `sift_up` operation after one comparison. Hence, its $O(\log N)$ worst-case bound doesn't guarantee that every update takes expensive operations. However, for a large decrease, since every target is set to the new minimum, binary movement towards the root is inevitable unless the targeted node is the root. This might be the reason why the Binary Heap still could beat the Fibonacci Heap during small decreases, but not during large decreases. On the other hand, for a Fibonacci Heap, it can just cut a child directly, insert it in the root list, and update its minimum pointer during large decreases. Additionally, no extraction process is done during the update phases in the experimental runs. Hence, consolidation is not triggered in the Fibonacci Heap. Furthermore, if the nodes are constantly added to the root list, the probability of a node in the root list being targeted again by the system is high. Even if the node in the root list is targeted for a `decrease_key` operation, the nodes are no longer cut repeatedly if they already are in the root list. This might be the reason why the Fibonacci Heap was more favored during the large-decrease experiment.

On the other hand, from experiment 2.3.1, it can be interpreted that the Fibonacci Heap still has to spend a great amount of time on insertion and extraction. This might be the reason why the Fibonacci Heap still can't beat the Binary Heap in complete workload performance due to the time spent in insertion and extraction processes outweighing its update savings. Separate node allocations, pointer traversal, and temporary consolidation storage might also be other contributors to why the Fibonacci Heap still can't beat the Binary Heap in complete workload performance.

2.6.2 Limitations

The following are the limitations faced during this empirical study.

- All results received in this study only cover the mentioned implementation on a single machine, three sizes, three seeds, and two update ratios.
- Experiments 2.3.2 and 2.3.3 are not a direct representation of workloads where decreases and extractions are mixed in certain cases, such as shortest-path processing.
- Experiment 2.3.3 constantly injects every `decrease_key` operation with a new minimum, and this is not a direct representation of the application of heaps.
- Only one source of bias is reduced by alternating the heap order. Other biases, such as seed and configuration orders, are still constant. Hence, input variation and changing machine conditions can still cause bias.
- Memory usage is not measured by the timing alone, and it cannot prove asymptotic complexity.

3. What I Learned

3.1 Implementation and Invariants

Regarding implementation, I learned that the Indexed Binary Heap is easier to follow due to its use of two vectors and array indices. Since the Fibonacci Heap involves altering pointers and maintaining parent, child, and circular sibling pointers, degrees, and marks that I'm not familiar with, it is harder to keep track of during implementation.

One concrete invariant I learnt during this study is the structure and the details of how the two vectors in the Indexed Binary Heap worked. I learnt that it is crucial to update both the position records after a binary swap to prevent the decrease_key operation from changing the wrong items.

I also improved my knowledge of how a Fibonacci Heap works from the implementation. I was fairly surprised that consolidation was never done regardless of any quantity of decrease_key operations unless an extraction operation is called.

3.2 Theory Versus Practical Performance

I initially expected Fibonacci's decrease-key operations to be faster because of its better amortized bound. However, I learnt that this does not guarantee faster performance for every workload. Small decreases can stop after one parent comparison in the binary heap, so its $O(\log N)$ worst-case bound does not mean every decrease performs that much work.

From theory, I imagined large decreases should substantially make Fibonacci's performance better from how it operates, but I now also learnt that this time advantage is easily outweighed by the insertion and extraction operations that are normally done on the Fibonacci heap as well.

3.3 Mistakes and Changes to My Approach

Initially, I only planned Experiments 2.3.1 and 2.3.2. However, I slowly realized that the small decrease_key experiment used small adjustments only, leading to less upward movement or cuts. Comparing that to my initial hypothesis, I was intrigued and added another experiment: Experiment 2.3.3, while also keeping Experiment 2.3.2 for comparisons. I also learned to separate update time and total workload time while implementing the experiment algorithms to provide more information regarding the different processes. Additionally, I started alternating the execution orders to reduce bias while implementing the experiment algorithms. Furthermore, regardless of my initial plan to let the user run the operations and retrieve the CSV data files, I now have persistent data files saved for this report so that I can have a persistent dataset.

4. AI Use

4.1 Tools Used and Extent of Assistance

I used ChatGPT for most of the planning, brainstorming, and structural guidance. It was also used to generate a Python program to graph the data in the CSV files with the matplotlib library.

4.2 Tasks Assisted by AI

ChatGPT was used for the following tasks:

- generate README.md
- implementation guidance of the Fibonacci Heap,
- generation of code for baseline and benchmark algorithms,
- code debugging,
- explanation of any confusing topics,
- result plotting,
- and report structural guidance.

4.3 Specific AI Errors and How I Addressed Them

While working with ChatGPT, I faced multiple AI errors, and the following are two of them.

4.3.1 Proposed experimentation not strong enough

ChatGPT initially proposed the idea of Experiment 2.3.2 when I mentioned what I wanted to test. However, after we implemented this experimental algorithm. I called it out for creating an algorithm that is not strong enough, and the results are not aligned with the theoretical views I had. So, I came up with another experiment: Experiment 2.3.3, with the help of ChatGPT's implementation. I conducted another experiment.

4.3.2 Received zero-byte PNG images when asked to use the Matplotlib API

When I prompted ChatGPT to plot the results of my experimentation, it attempted to do so, but I ended up receiving zero-byte PNG images. I called it out for this and requested that it not only generate images but also generate a Python program so that I can generate the images myself as well.

4.4 What I Understand and What I Took on Trust

In the entire implementation, I understood the Fibonacci implementation well since I implemented it myself with partial guidance from ChatGPT. Even though I entirely retrieved the code for the

IndexedBinaryHeap implementation and benchmarking experimental algorithms, I have personally gone through all the code generated along with ChatGPT and now fully understand what each process is doing, either through past knowledge or explicit questioning and conversation with ChatGPT. Even though I asked ChatGPT to structurally plan my code project structure and report structure, I also understood what it was doing and cross-checked it with the specifications as well.

However, there are certain components in this assignment that I took on trust, even though they were generated by ChatGPT. I am still a bit uncertain about how CMake works and how it affects my implementation. I am only aware that I should have it to build the project and make the entire program easily runnable for the marker. Additionally, even though I asked ChatGPT to provide me with a Python program for the matplotlib function and to produce the result plot charts, I still haven't reviewed it properly enough to say that I understand it well. I also used the images generated again by ChatGPT and haven't run the program myself.